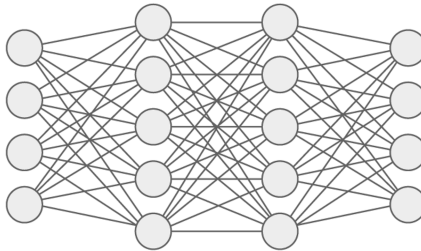


Deep Learning and Artificial Neural Networks

Optimization Algorithms



Contents

- 1 Importance of Optimization Algorithms for Neural Network
- 2 Mini-Batch Gradient Descent
- 3 Exponentially Weighted Averages
- 4 Gradient Descent with Momentum
- 5 RMSProps
- 6 Adam Optimization Algorithm
- 7 Learning Rate Decay
- 8 Local Optima Problem

Optimization Algorithms for Neural Networks

- Applying machine learning involves a **highly iterative** and empirical approach.
- We often need to train **many models** to find one that works well.
- Deep Learning performs best with **big data**, but large datasets can slow training.
- **Fast and effective optimization algorithms** are crucial for speeding up the training process.
- Speeding up training improves overall team efficiency, enabling quicker experimentation and iteration.

Mini-Batch Gradient Descent

- **Vectorization:** Processes all training examples without explicit loops.
- **Large Datasets:** Training on huge datasets (e.g., 5M examples) with full-batch gradient descent is slow.
- **Mini-Batches:** Split the full training set into smaller subsets, so called mini-batches (e.g., 1,000 examples per mini-batch).
- **Multiple Updates:** Each mini-batch is used to perform a gradient descent step; one epoch can include thousands of steps.
- **Efficiency:** Allows weight updates to begin before processing the entire dataset, speeding up convergence.

Notation and Data Organization

- Let $X^1, X^2, \dots, X^M$ be the training inputs and $Y^1, Y^2, \dots, Y^M$ the outputs.
- With mini-batches, split the data as follows:
 - For a training set of 5M examples and mini-batches of 1,000 examples, you get 5,000 mini-batches.
 - Denote the T -th mini-batch as $X^{\{T\}}$ and $Y^{\{T\}}$.
- Other indexing conventions:
 - $X^{(i)}$: the i -th training example.
 - $Z^{[l]}$: the input to the activation at layer l of the neural network.

Process of Mini-Batch Gradient Descent

1 Forward Propagation:

- For mini-batch T , compute $Z^{[1]} = W^{[1]}X^{\{T\}} + b^{[1]}$, then apply the activation function $A^{[1]} = G^{[1]}(Z^{[1]})$, and so on.

2 Cost Computation:

- Compute the cost $J^{\{T\}}$ on the mini-batch (using a denominator of 1,000).

3 Backpropagation:

- Compute gradients for the mini-batch and update weights.

4 Iteration:

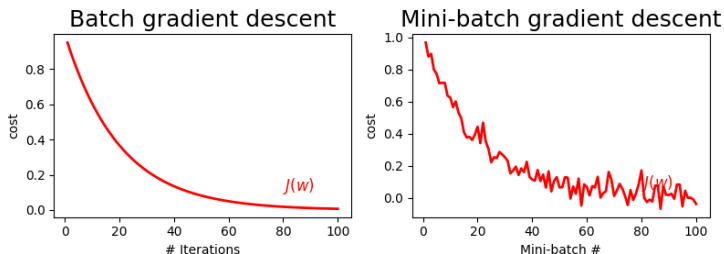
- Process all mini-batches (e.g., $T = 1$ to 5000) within one epoch.

Advantages of Mini-Batch Gradient Descent

- Speeds up training significantly for large datasets.
- Allows more frequent updates to model parameters.
- Reduces memory requirements compared to batch gradient descent.
- Widely used in deep learning for its efficiency.

Training with Mini-Batch Gradient Descent

Mini-Batch vs. Batch Gradient Descent

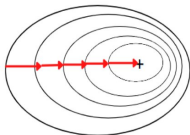


- **Batch Gradient Descent (left):** The cost function decreases smoothly. Smooth descent, slower updates.
- **Mini-Batch Gradient Descent (right):** The cost function fluctuates but trends downward. Noisy descent, faster convergence.
- Different mini-batches contain different samples, causing slight variations in gradient direction at each step.
- Despite fluctuations, mini-batch gradient descent converges faster due to more frequent updates.

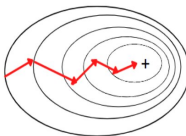
Gradient Descent Variants

- **Batch Gradient Descent:** Uses the entire dataset for each update.
- **Stochastic Gradient Descent (SGD):** Uses one example per update.
- **Mini-Batch Gradient Descent:** Uses a small batch per update (hybrid approach).

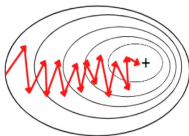
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



Choosing the Mini-Batch Size

- **Batch Size = Total Training Set** → Batch Gradient Descent.
- **Batch Size = 1** → Stochastic Gradient Descent.
- **Best practice:** Choose **64, 128, 256, 512, 1024**.
- Ensure the batch **fits in CPU/GPU memory**.

Exponentially Weighted Averages

- We will explore faster optimization algorithms than the standard gradient descent.
- These algorithms leverage a concept known as **exponentially weighted averages** (also referred to as **exponentially weighted moving averages** in statistics).
- Before delving into these advanced optimization techniques, let us first understand the mechanics of exponentially weighted averages.

Exponentially Weighted Averages (EWAs)

- EWAs calculate the average of a sequence while giving more weight to recent values.

- **Formula:**

$$\theta_t = \beta \cdot \theta_{t-1} + (1 - \beta) \cdot v_t$$

where:

- θ_t : Exponentially weighted average at time t .
- v_t : Value at time t .
- β : Smoothing factor ($0 < \beta < 1$).
- **Key characteristics:**
 - Recent values have higher weights; older values have exponentially decreasing weights.
 - Bias correction for small t : To correct for the initial bias when starting with $v_0 = 0$, we use:
$$\hat{\theta}_t = \frac{\theta_t}{1 - \beta^t}$$

This helps account for the fact that at the beginning, the average is systematically lower than it should be.
 - Effective window size depends on β :
 - $\beta = 0.9$: 10 past values.
 - $\beta = 0.98$: 50 past values.

Exponentially Weighted Averages (EWAs)

Application in optimization: Used in optimization algorithms like **Momentum**, **RMSprop** and **Adam** to smooth noisy gradients. Leads to faster convergence during training.

Example calculation: Temperature in Celsius: Suppose we are tracking daily temperatures in Celsius over three days, and we want to calculate the exponentially weighted average temperature with $\beta = 0.9$:

- Day 1 (v_1): 20°C
- Day 2 (v_2): 22°C
- Day 3 (v_3): 18°C

$$\theta_t = \beta \cdot \theta_{t-1} + (1 - \beta) \cdot v_t$$

1 Initialize $\theta_0 = 0$.

2 Calculate θ_1 :

$$\theta_1 = 0.9 \cdot 0 + 0.1 \cdot 20 = 2.0$$

3 Calculate θ_2 :

$$\theta_2 = 0.9 \cdot 2.0 + 0.1 \cdot 22 = 1.8 + 2.2 = 4.0$$

4 Calculate θ_3 :

$$\theta_3 = 0.9 \cdot 4.0 + 0.1 \cdot 18 = 3.6 + 1.8 = 5.4$$

Result: $\theta_1 = 2.0$, $\theta_2 = 4.0$, $\theta_3 = 5.4$.

Exponentially Weighted Averages (EWAs)

Example Calculation: Temperature in Celsius with Bias Correction

1 Initialize $\theta_0 = 0$.

2 Calculate θ_1 :

$$\theta_1 = 0.9 \cdot 0 + 0.1 \cdot 20 = 2.0$$

Apply bias correction:

$$\hat{\theta}_1 = \frac{\theta_1}{1 - \beta^1} = \frac{2.0}{1 - 0.9} = \frac{2.0}{0.1} = 20.0$$

3 Calculate θ_2 :

$$\theta_2 = 0.9 \cdot 2.0 + 0.1 \cdot 22 = 1.8 + 2.2 = 4.0$$

Apply bias correction:

$$\hat{\theta}_2 = \frac{\theta_2}{1 - \beta^2} = \frac{4.0}{1 - 0.9^2} = \frac{4.0}{1 - 0.81} = \frac{4.0}{0.19} \approx 21.05$$

4 Calculate θ_3 :

$$\theta_3 = 0.9 \cdot 4.0 + 0.1 \cdot 18 = 3.6 + 1.8 = 5.4$$

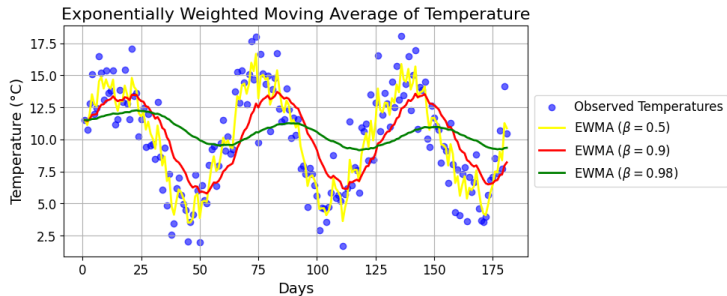
Apply bias correction:

$$\hat{\theta}_3 = \frac{\theta_3}{1 - \beta^3} = \frac{5.4}{1 - 0.9^3} = \frac{5.4}{1 - 0.729} = \frac{5.4}{0.271} \approx 19.93$$

Result with bias correction: $\hat{\theta}_1 = 20.0$, $\hat{\theta}_2 \approx 21.05$, $\hat{\theta}_3 \approx 19.93$.

Exponentially Weighted Averages (EWAs)

Day (t)	Temperature (θ_t)	
1	40°F	(4°C)
2	49°F	(9°C)
3	45°F	(7°C)
$\vdots$	$\vdots$	$\vdots$
181	60°F	(15°C)



Importance of β in Exponentially Weighted Averages

The parameter β in the exponentially weighted moving average (EWMA) determines how much weight is given to past values versus the current value. It controls the "memory" of the system and influences how smooth the moving average is:

- **High β (e.g., 0.98):** Assigns more weight to past values, making the moving average smoother but slower to react to recent changes.
- **Low β (e.g., 0.5):** Gives more weight to recent values, making it more responsive to fluctuations but also noisier.

By comparing different values of β in the plot, we can observe:

- **$\beta = 0.5$ (Yellow Line):** Reacts quickly to fluctuations in temperature. More sensitive to noise in the data. The moving average follows the observations more closely but does not smooth out short-term variations effectively.
- **$\beta = 0.9$ (Red Line):** Provides a balanced smoothing effect. Captures the general trend while filtering out some noise. Commonly used in practical applications for stable updates.
- **$\beta = 0.98$ (Green Line):** Smooths the curve significantly. Responds very slowly to changes, making it less influenced by short-term fluctuations. Useful when we want a long-term trend without reacting too much to temporary spikes.

Choosing the right β : A moderate β (e.g., 0.9) is often a good compromise between stability and responsiveness. In deep learning optimization, a similar principle applies when using momentum-based optimization algorithms, where β helps balance between stability and adaptability.

Gradient Descent with Momentum

Gradient Descent with Momentum (often simply called "**Momentum**") typically converges faster than standard Gradient Descent.

The key idea behind Momentum is to compute a weighted average of past gradients, giving more importance to recent gradients, and then use this averaged gradient to update the model's weights. This helps the optimization process move more efficiently toward the minimum by reducing oscillations and accelerating convergence in relevant directions.

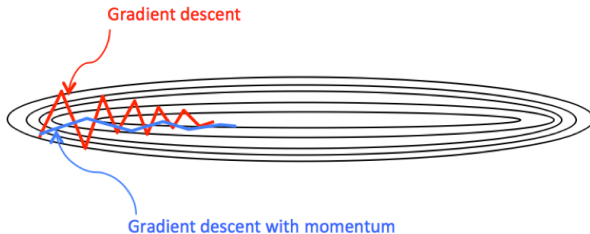
Gradient Descent with Momentum

Consider minimizing a cost function with an elliptical shape, as depicted below. Standard gradient descent oscillates while trying to reach the minimum. To prevent these oscillations (overshooting), we typically use a small learning rate, which slows down convergence.

Ideally, we want:

- Slow learning on the vertical axis (to reduce oscillations).
- Faster learning on the horizontal axis.

Good news: This is exactly what *gradient descent with momentum* achieves!



Momentum-Based Weight Update

Recall EWA formula: $\theta_t = \beta \cdot \theta_{t-1} + (1 - \beta) \cdot v_t$

On every iteration t , we compute the exponentially weighted moving average of the gradients:

$$V_{dW} = \beta V_{dW} + (1 - \beta) dW \quad (1)$$

$$V_{db} = \beta V_{db} + (1 - \beta) db \quad (2)$$

- V_{dW} is the velocity term associated with the weight updates (W), and V_{db} is the velocity term associated with the bias updates (b).
- β is a hyperparameter that controls the contribution of past gradients.
- dW , db are the current gradients with respect to the weights W and biases b , respectively.

We then update the parameters using the computed moving averages:

$$W \leftarrow W - \alpha V_{dW} \quad (3)$$

$$b \leftarrow b - \alpha V_{db} \quad (4)$$

where α is the learning rate.

Intuition: Rolling Ball Analogy

The cost function can be visualized as a bowl, and a ball rolling towards the minimum:

- The derivatives dW and db represent the **acceleration** of the ball.
- The terms V_dW and V_db represent the **velocity** of the ball.
- The parameter β represents **friction**, controlling how much momentum the ball gains as it moves toward the minimum.

Effect: Momentum smooths out the updates, reduces oscillations, and speeds up convergence.

RMSProp

- RMSProp (Root Mean Square Propagation) is an adaptive optimization algorithm.
- RMSProp can also speed up gradient descent.
- Works by maintaining an exponentially weighted moving average of squared gradients.

The RMSProp optimization technique updates parameters as follows:

$$Sd_W = \beta Sd_W + (1 - \beta)d_W^2 \quad (5)$$

$$Sd_b = \beta Sd_b + (1 - \beta)d_b^2 \quad (6)$$

$$W = W - \frac{\alpha}{\sqrt{Sd_W + \epsilon}} d_W \quad (7)$$

$$b = b - \frac{\alpha}{\sqrt{Sd_b + \epsilon}} d_b \quad (8)$$

where:

- S is the exponentially weighted moving average of squared gradients.
- d_W is the gradient at time step t .
- β (typically 0.9) controls the decay rate.
- α is the learning rate.
- ϵ (small value) prevents division by zero.

Adam Optimization Algorithm

- Adam (Adaptive Moment Estimation) is an advanced gradient-based optimization algorithm.
- Combines ideas from **Momentum** and **RMSProp** to adapt learning rates.
- Widely used in deep learning due to its fast convergence and robustness. Usually, the default choice for the optimization algorithm.

Why Adam? Issues with Previous Methods

- **Gradient Descent** struggles with choosing the right learning rate.
- **Momentum** helps smooth updates but can overshoot.
- **RMSProp** adapts learning rates but does not account for past gradients.
- **Adam** combines both benefits to improve training speed and stability.

Adam Optimization Algorithm

"Momentum"-like update (First Moment Estimate):

$$V_{d_w} = \beta_1 V_{d_w} + (1 - \beta_1) d_w \quad (9)$$

$$V_{d_b} = \beta_1 V_{d_b} + (1 - \beta_1) d_b \quad (10)$$

- V_{d_w}, V_{d_b} : Exponentially weighted averages of gradients (first moments).
- β_1 : Decay rate for the first moment estimate (typically 0.9).
- d_w, d_b : Gradients of the loss function with respect to weights w and biases b .

"RMSprop" (Second Moment Estimate):

$$S_{d_w} = \beta_2 S_{d_w} + (1 - \beta_2) d_w^2 \quad (11)$$

$$S_{d_b} = \beta_2 S_{d_b} + (1 - \beta_2) d_b^2 \quad (12)$$

- S_{d_w}, S_{d_b} : Exponentially weighted averages of squared gradients (second moments).
- β_2 : Decay rate for the second moment estimate (typically 0.999).

Bias Correction:

$$V_{d_w}^{\text{corrected}} = \frac{V_{d_w}}{1 - \beta_1^t}, \quad V_{d_b}^{\text{corrected}} = \frac{V_{d_b}}{1 - \beta_1^t} \quad (13)$$

$$S_{d_w}^{\text{corrected}} = \frac{S_{d_w}}{1 - \beta_2^t}, \quad S_{d_b}^{\text{corrected}} = \frac{S_{d_b}}{1 - \beta_2^t} \quad (14)$$

- Bias correction adjusts the estimates to account for their initialization at zero.
- t : Current iteration number.

Adam Optimization Algorithm

Parameter Update:

$$w := w - \alpha \frac{V_{d_w}^{\text{corrected}}}{\sqrt{S_{d_w}^{\text{corrected}} + \epsilon}} \quad (15)$$

$$b := b - \alpha \frac{V_{d_b}^{\text{corrected}}}{\sqrt{S_{d_b}^{\text{corrected}} + \epsilon}} \quad (16)$$

Here:

- α : Learning rate (controls step size).
- ϵ : Small constant (e.g., 10^{-8}) to prevent division by zero.

Adam Optimization Algorithm: Hyperparameters

Adam has several hyperparameters that control its behavior. Below is a list of these hyperparameters along with their typical default values:

- **Learning Rate (α):**

- Controls the step size during updates.
- Default value: $\alpha = 0.001$.

- **First Moment Decay Rate (β_1):**

- Governs the exponential decay rate for the first moment estimate (momentum term).
- Default value: $\beta_1 = 0.9$.

- **Second Moment Decay Rate (β_2):**

- Governs the exponential decay rate for the second moment estimate (RMSprop term).
- Default value: $\beta_2 = 0.999$.

- **Epsilon (ϵ):**

- A small constant to prevent division by zero in the update rule.
- Default value: $\epsilon = 10^{-8}$.

Learning Rate Decay

What is Learning Rate Decay?

- Learning Rate Decay refers to the process of gradually reducing the learning rate during training.
- It helps the optimization algorithm converge more effectively by allowing larger steps initially and smaller steps later.

Why Use Learning Rate Decay?

- Prevents overshooting in early stages when the learning rate is high.
- Allows finer adjustments as the model approaches the minimum.
- Improves convergence speed and final model performance.

Learning Rate Decay

Common Types of Learning Rate Decay:

- **Step Decay:** Reduce the learning rate by a factor after a fixed number of epochs.

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/d \rfloor}$$

where α_0 : initial learning rate, γ : decay factor, d : step size (epochs).

- **Exponential Decay:** Decrease the learning rate exponentially over time.

$$\alpha_t = \alpha_0 \cdot e^{-kt}$$

where k : decay rate, t : current epoch.

- **1/t Decay:** Inverse time-based decay.

$$\alpha_t = \frac{\alpha_0}{1 + kt}$$

where k : decay rate, t : current epoch.

Exponential Learning Rate Decay Example

Exponential Learning Rate Decay Formula:

$$\alpha_t = \alpha_0 \cdot e^{-kt}$$

Where:

- $\alpha_0 = 0.1$ (initial learning rate),
- $k = 0.1$ (decay rate),
- t : Epoch number.

Example of exponential decay over epochs

Epoch (t)	Learning Rate (α_t)
0	0.100
1	0.090
2	0.082
3	0.074
4	0.067
5	0.061
10	0.037
15	0.022
20	0.013

Note: The learning rate decreases exponentially as the number of epochs increases.

Local Optima in Neural Networks

- Neural network loss landscapes can have **many local minima**.
- Traditional gradient descent may get stuck in poor local optima.
- **Saddle points**: These are points where the gradient is zero, but they are not local minima or maxima. Instead, they represent flat regions in some dimensions and steep slopes in others. Saddle points can slow down optimization because gradients in flat directions become very small, making progress difficult.
- Advanced optimizers like Adam, RMSProp, and Momentum help escape saddle points and local minima by incorporating momentum or adaptive learning rates.
- While bad local optima are rare in high-dimensional spaces, **plateaus** can significantly slow down training.
- **Plateaus**: These are regions of the loss landscape where the gradient is close to zero across all dimensions, leading to very small updates. This can cause the optimization process to stall for many iterations.
- Gradient updates become **very small** in flat regions (e.g., plateaus or near saddle points), which can hinder convergence.
- Adaptive learning rates (e.g., in Adam or RMSProp) and proper initialization techniques (e.g., He or Xavier initialization) help mitigate these issues by ensuring that updates remain effective even in challenging regions of the loss landscape.